SQL Server 2025 Vectors

Under the Hood and Into Production

Anthony Nocentino



Anthony E. Nocentino

Senior Principal Field Solution Architect, Everpure · Global Database Lead

What I work on

- System architecture and performance
- SQL Server, Kubernetes, and containers
- Microsoft Data Platform MVP
- Pluralsight author



nocentino.com · github.com/nocentino · anocentino@everpuredata.com

Agenda

1 Vectors 101: the math you need

2 Inside the VECTOR type

3 **Demo 1: vector internals**

4 Vector indexes: where they help and where they don't

5 **Demo 2: index performance at scale**

6 Into production: patterns and the app

7 **Demo 3: the RAG application**

8 Takeaways, then questions

Before we start

Level 300: you know SQL Server internals. You don't need any machine learning.

You already know

- Reading execution plans
- Pages, B-tree indexes, the buffer pool
- Filegroups, the log, and backups
- Stored procedures and permissions

We'll build from here

- What an embedding is
- Distance: cosine, dot, Euclidean
- Exact vs approximate search, and recall
- DiskANN, at the depth a DBA needs

You don't need

- Machine learning or linear algebra
- A cloud account or API keys
- Python: the app is a tour, not a lesson

Get the code: github.com/nocentino/vector-under-the-hood

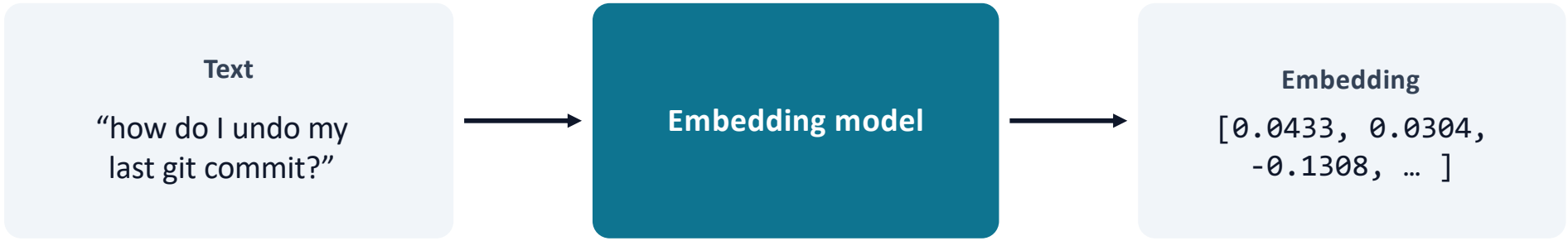
Vectors 101

The math you need to know



From text to a vector

Meaning in, a fixed-length list of numbers out. Three ideas carry the rest of the session.



A vector is a list of numbers

An embedding model turns text into dimensions. In our case, 768

- Stored in a VECTOR(N) column
- VECTOR(768)
- The model picks the count, not you
- Max N: 1,998 (float16: 3,996)

Direction is meaning

Text with similar meaning points the same way.

- Our model's vectors all have length 1
- So only the angle between them matters

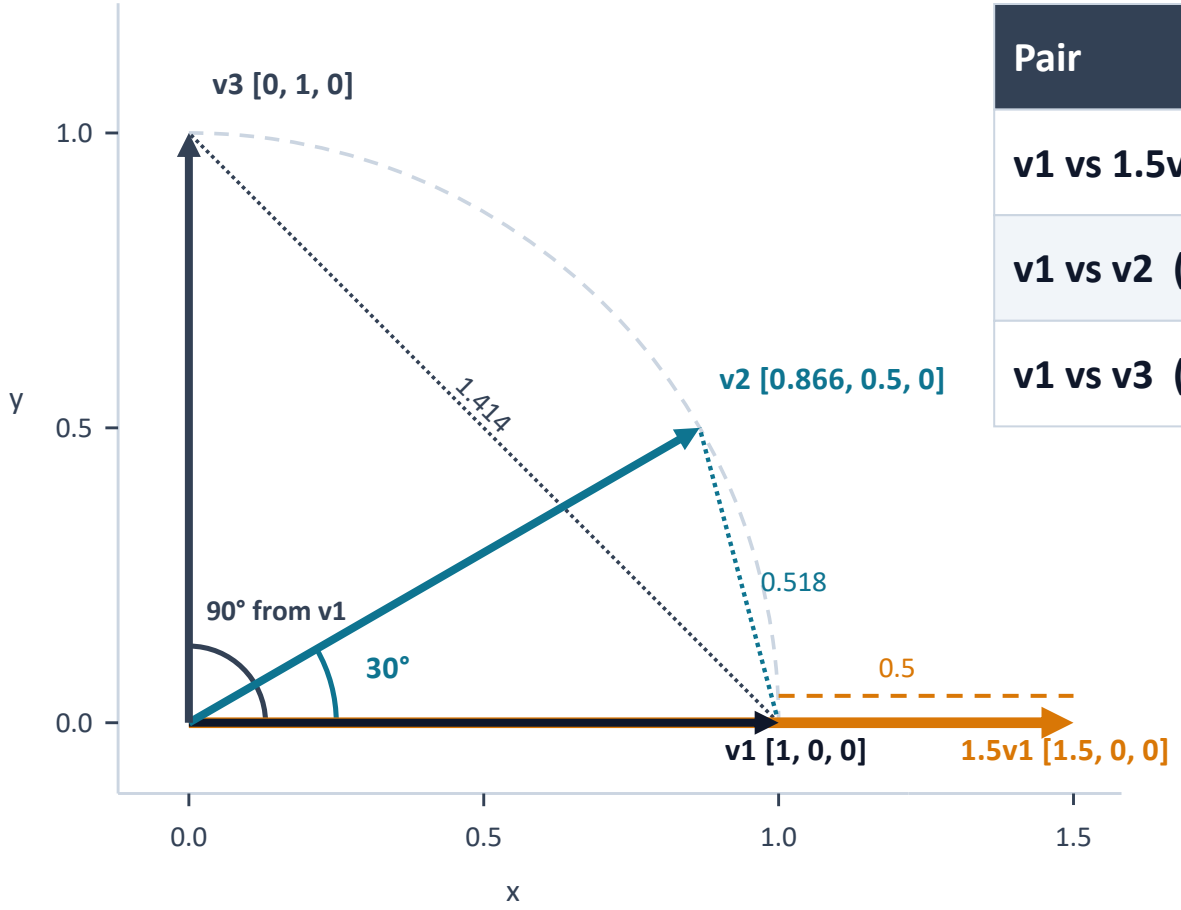
Search is a sort by distance

Embed the question, measure, keep the closest K.

- VECTOR_DISTANCE: lower is closer
- Strong match ~0.12
- Unrelated ~0.68

Three vectors, three distances

Lower is always closer. For text, use cosine; dot and Euclidean matter only when length means something.



Pair	cosine distance	euclidean	dot
v1 vs 1.5v1 (0°)	0	0.5	−1.5
v1 vs v2 (30°)	0.134	0.518	−0.866
v1 vs v3 (90°)	1	1.414	0

VECTOR_DISTANCE flips similarity so lower is closer:
cosine distance = 1 − cos θ

cos θ (similarity)	1 − cos θ (distance)
cos 0° = 1	**1 − 1 = 0**
cos 30° = 0.866	**1 − 0.866 = 0.134**
cos 90° = 0	**1 − 0 = 1**

Cosine, Euclidean, or dot?

For text embeddings, use cosine. The others matter only when a vector's length means something.

Cosine

Use for text. The default.

- Compares direction, ignores length
- Short and long text on one topic score alike

Dot

Use when length is a signal.

- Recommenders: length = popularity
- More people reviewed a pizza place

Euclidean

Use when absolute values matter.

- Sensors, images, clustering
- A sensor running twice as hot: same direction, far away

Use the metric the model's docs name and build the index with the same one.

Inside the VECTOR type

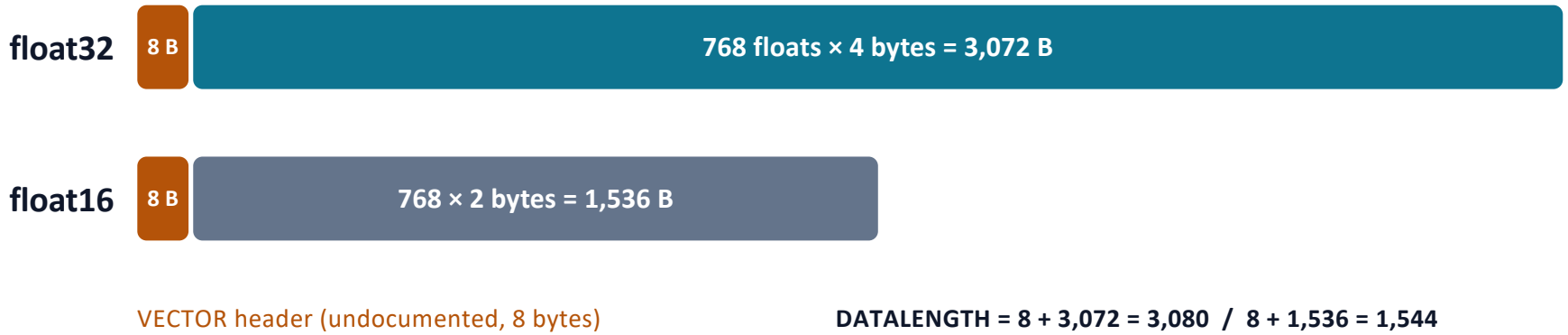
Storage layout, precision, and exact search



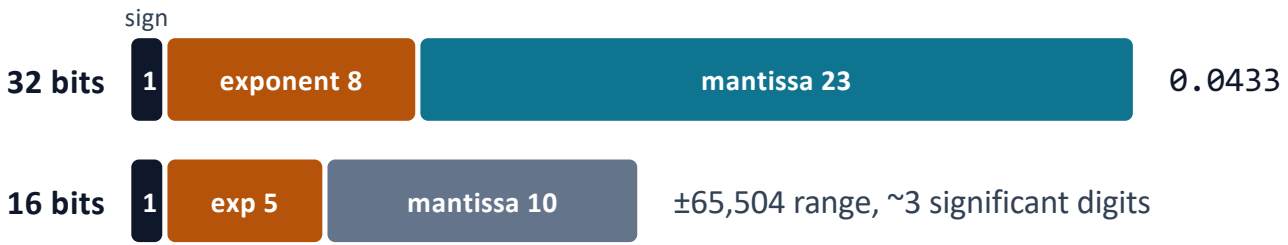
Anatomy of a VECTOR

IEEE 754 floats behind an 8-byte header, stored in the row

One VECTOR(768) value: an 8-byte header, then the floats



One element, bit by bit (IEEE 754, little-endian on disk)



Dimensions set the size

VECTOR(n) stores n numbers behind the same 8-byte header

Embedding model	Dimensions	VECTOR(n) float32	VECTOR(n, float16)
nomic-embed-text (our demos)	768	3,080 B	1,544 B
mxbai-embed-large	1,024	4,104 B	2,056 B
OpenAI text-embedding-3-small	1,536	6,152 B	3,080 B
OpenAI text-embedding-3-large	3,072	12,296 B won't fit (Msg 2717)	6,152 B
The limit	1,998 / 3,996	8,000 B	8,000 B

The cap is 8,000 bytes, not a dimension count: 1,998 four-byte numbers or 3,996 two-byte ones. That's why float16 fits twice the dimensions.

Measured with DATALENGTH on SQL Server 2025 CU10. Size grows in a straight line with dimensions.

float32 or float16?

Use float16 when storage or dimensions are the limit and your vectors are unit length. Otherwise float32.

float32 (the default)

4 bytes per element, up to 1,998 dimensions.

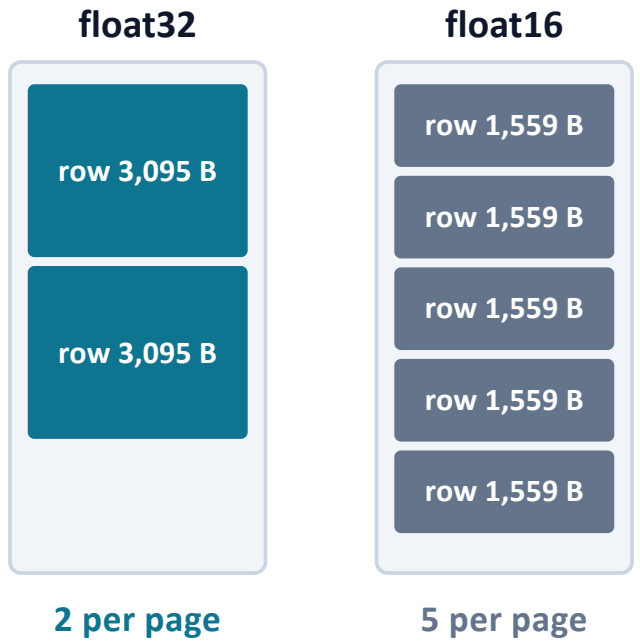
- Generally available in SQL Server 2025
- Full precision for any workload
- Every client driver handles it

float16 (preview)

2 bytes per element, up to 3,996 dimensions.

- Half the storage and I/O; smaller memory footprint
- Fits 3,072-d models
- Needs PREVIEW_FEATURES; binary transport needs ODBC 18.7.1+ or JDBC 13.4+
- Can't mix with float32 in VECTOR_DISTANCE

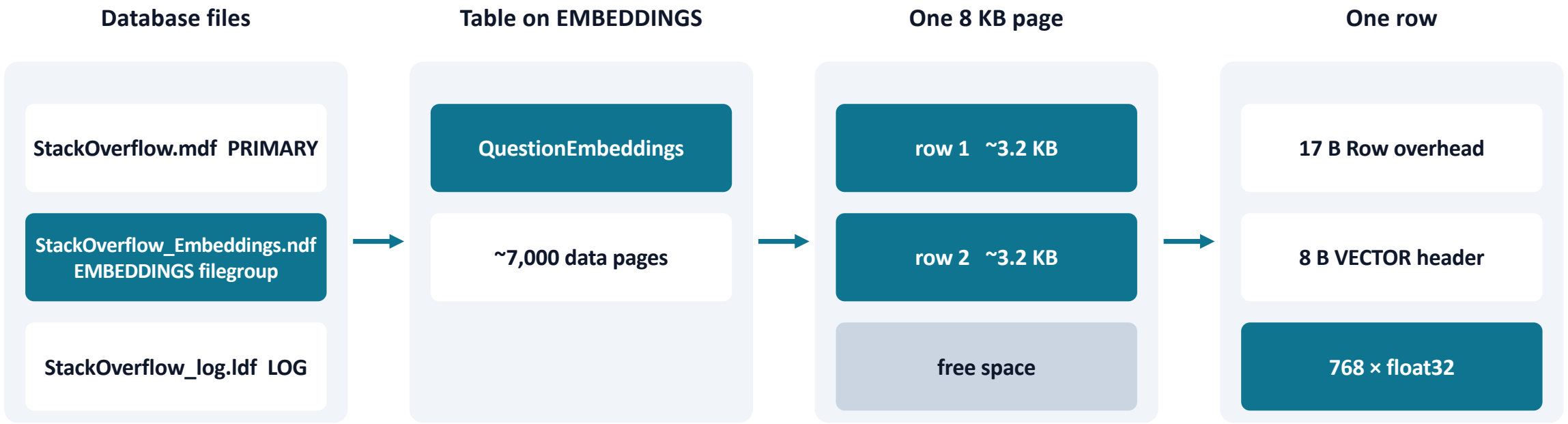
768 dims on one 8 KB page



Same page, more vectors: 1,000 vectors at 768 dims in a test table with only an INT key and the vector took 500 pages as float32 and 200 as float16. An exact scan reads 60% fewer pages.

Where the bytes live

From database file, to filegroup, to an 8 KB page, to the row



~1.5 rows per page, so an exact scan of 10,000 vectors reads about 7,000 pages.
17 B row overhead: the row header plus an offset array for the two variable-length columns.
Some pages hold 1 row, some 2: rows were loaded out of key order, and page splits left pages half full.

Decode those 3,072 bytes by hand and all 768 floats match what SELECT prints.

Why a DBA cares about this data

Its more data to store, read, backup, log, index…

Buffer pool

- An exact search reads every vector page
- 1M rows: ~535,000 pages (4.1 GB) per query

Scans

- Distance needs every row; no predicate skips one
- Cost grows with every row you add

Indexes

- A vector can't be a B-tree key, compared, or sorted
- Only a vector index (a graph) helps

Backups

- Floats are hard to compress
- ZSTD 1.41:1 with vectors, 1.88:1 without

Compression

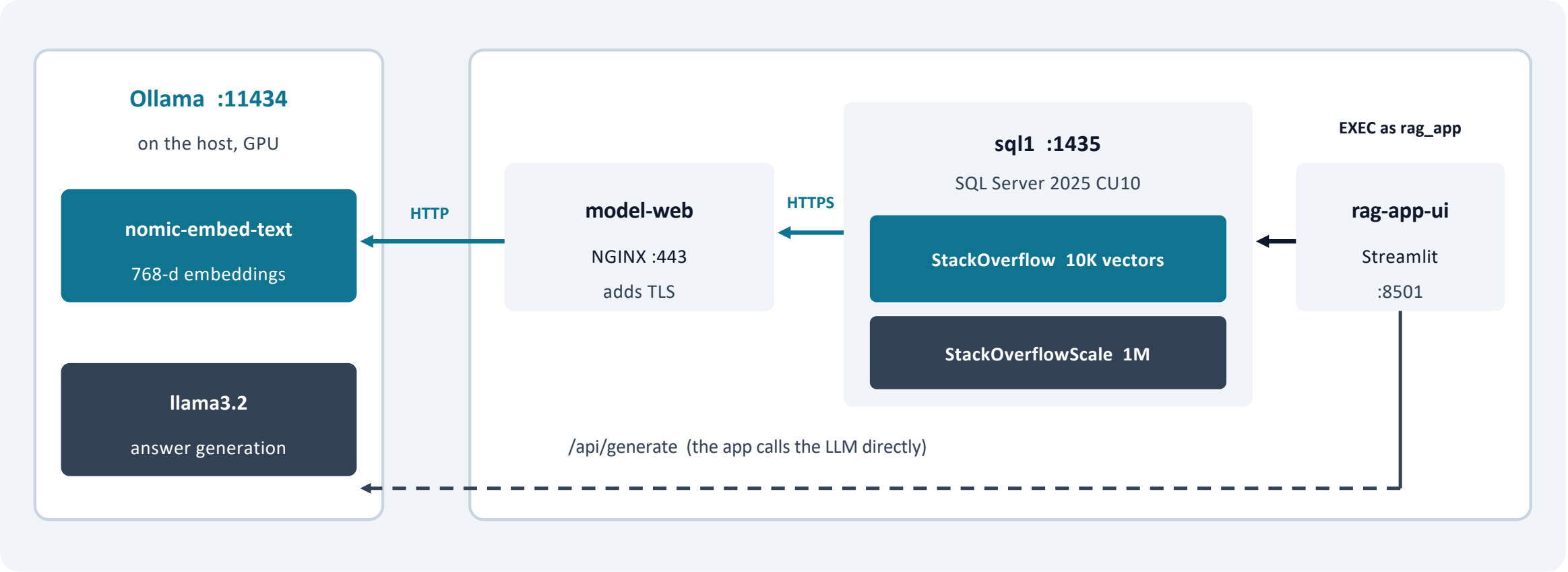
- ROW and PAGE compression save nothing
- 2,000 vectors: 1,021 pages, 1,024 with PAGE

The log

- ~3.6 KB of log per inserted row
- 1M rows: ~3.6 GB of log per load

The test lab

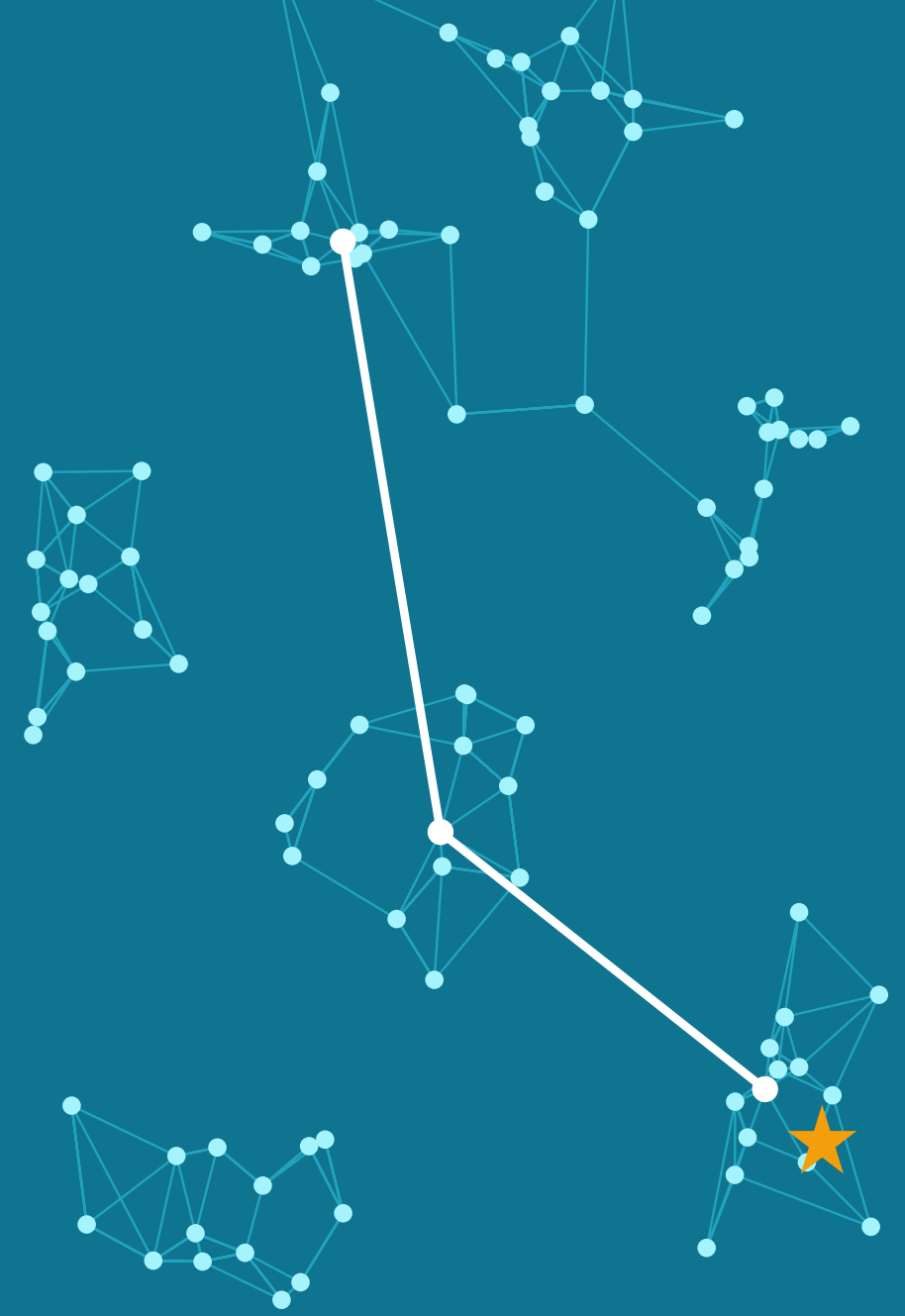
One laptop: SQL Server 2025 CU10 in Docker, Ollama on the host, no cloud



LIVE DEMO

Demo 1: Vector internals

Files and pages · decode a vector off the page · float16 · distance · the exact-search plan



Vector indexes

Where they help, and where they don't



Exact kNN vs approximate ANN

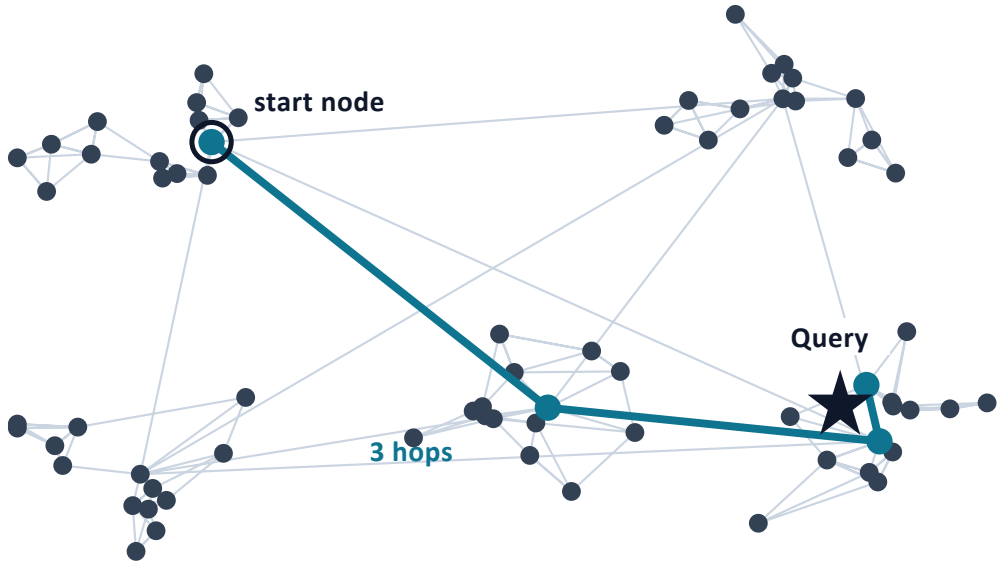
Exact visits every vector. ANN walks a graph and visits a few.

Exact kNN: measure every vector



Every row, every time. O(n). Perfect recall.

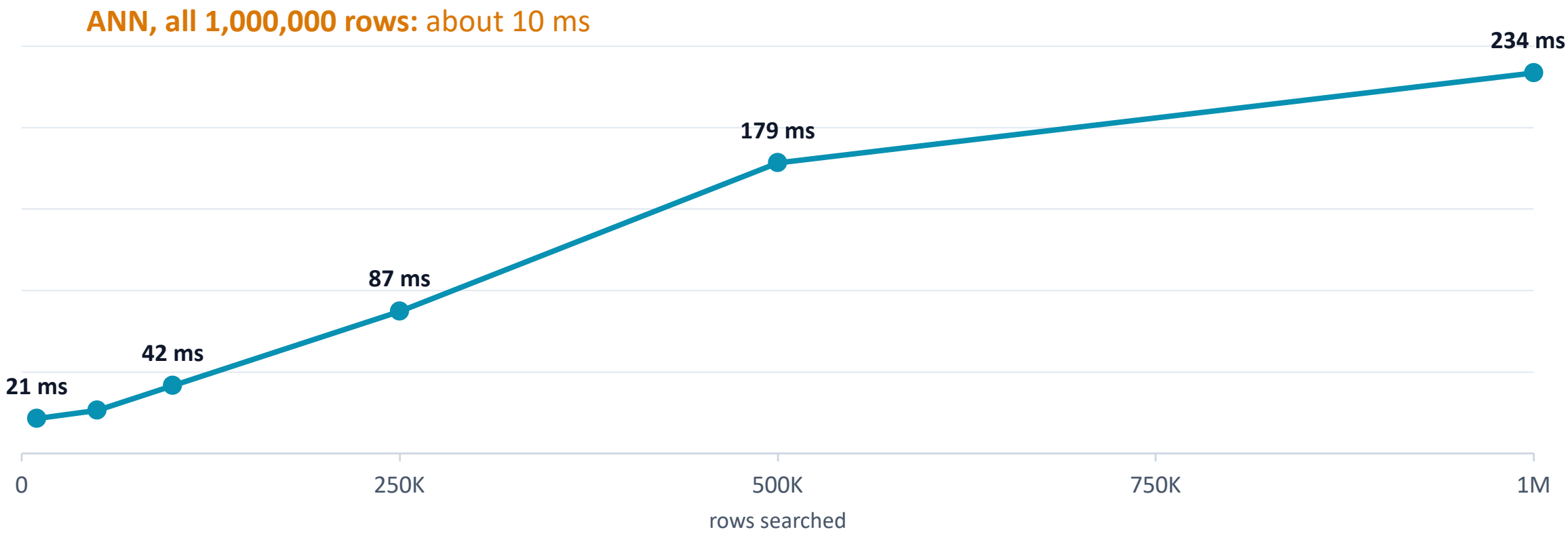
ANN (DiskANN): walk a graph



A few hops toward the query. Far fewer reads. Approximate.

Exact search grows with every row

One exact kNN query on this laptop: average elapsed time by rows searched. Watch the shape, not the ms.



Times are from one laptop and move from run to run. Compare them to each other, not to your servers. Logical reads don't change.

Recall: the price of approximate

How many of the true top 10 the index found, at 1,000,000 rows

Search	Recall@10	Time per query	Logical reads (avg)
ANN, TOP_N = 10	0.84 – 0.89	8 – 15 ms	~6,400
ANN, TOP_N = 50, keep 10	0.96 – 0.98	24 – 28 ms	~20,800
Exact kNN	1.0	175 – 265 ms	535,731

Times are from one laptop and move from run to run. Compare them to each other, not to your servers. Logical reads don't change.

kNN or ANN: which, when

Microsoft's guidance: exact search is fine up to about 50,000 vectors

Situation	Use	Why
Under ~50,000 vectors, or a filter cuts it that far	Exact kNN	Fast enough, perfect recall, nothing to maintain
A small table (about 1,000 rows)	Exact kNN	The index costs more reads than the scan
Hundreds of thousands to millions of vectors	ANN	535,731 vs 5,564 reads for one question at 1M rows
You need every true neighbor	Exact kNN	ANN found 84–89% of the true top 10 at 1M rows
A selective filter, on SQL Server 2025	kNN, or ANN with TOP_N ≥ K ÷ pass rate	Filters run after the walk; a small TOP_N can return 0 rows
The table changes all day	kNN, or ANN on Azure SQL	The 2025 index makes the table read-only

Why the selective filter matters: the index returns TOP_N candidates, then WHERE runs. If 1 row in 500 passes your filter, TOP_N = 5 returns 0 rows with no error, and the app says “no results” while the rows exist. Recall measures the same loss: the share of the true top K the index finds (0.84–0.89 at TOP_N = 10, 0.96+ at 50). Oversample to get it back.

Limits on SQL Server 2025, and what Azure SQL changes

The index is preview on 2025 and GA on Azure SQL, where the latest version removes all three limits.

	SQL Server 2025: earlier index, preview	Azure SQL/MI/SQL in Fabric: latest index, GA
Writes	**Read-only table** INSERT, UPDATE, DELETE fail with Msg 42231. Loading = drop the index, insert, rebuild. Demo 3 runs that cycle.	**Full DML** Changes join the graph in the background. sys.dm_db_vector_indexes shows how far behind it is.
Filters	**Walk, then filter** `VECTOR_SEARCH(…, TOP_N = 5) … WHERE p.Score > 5000` The walk returns 5, then WHERE runs: 0 rows, no error. You guess TOP_N: 2,500 gets 5 back.	**Filter during the walk** `SELECT TOP (5) WITH APPROXIMATE … WHERE p.Score > 5000` Keeps walking until 5 rows qualify. No TOP_N to guess.
Syntax	**TOP_N, deprecated** This earlier index version will be retired.	**TOP (N) WITH APPROXIMATE** TOP_N fails with Msg 42274. The optimizer picks ANN or kNN; FORCE_ANN_ONLY overrides it.

Keep vector queries in procs, and moving to the new syntax happens in one place.

What an index build costs you

Building the index over 100,000 vectors on 12 cores. MAXDOP trades build time for room on the box.

Build	Elapsed	CPU	Memory grant	Exact kNN during the build
Default (all 12 cores)	140 s	~1,520 s	121 MB	~800 ms
WITH (MAXDOP = 4)	237 s	914 s	63 MB	~250 ms
WITH (MAXDOP = 1)	908 s	885 s	31 MB	~110 ms

Reads keep working

- SELECT and exact kNN run, starved for CPU

Writes and ANN don't

- INSERT, UPDATE fail with Msg 42231 at once
- VECTOR_SEARCH: Msg 42227 until done

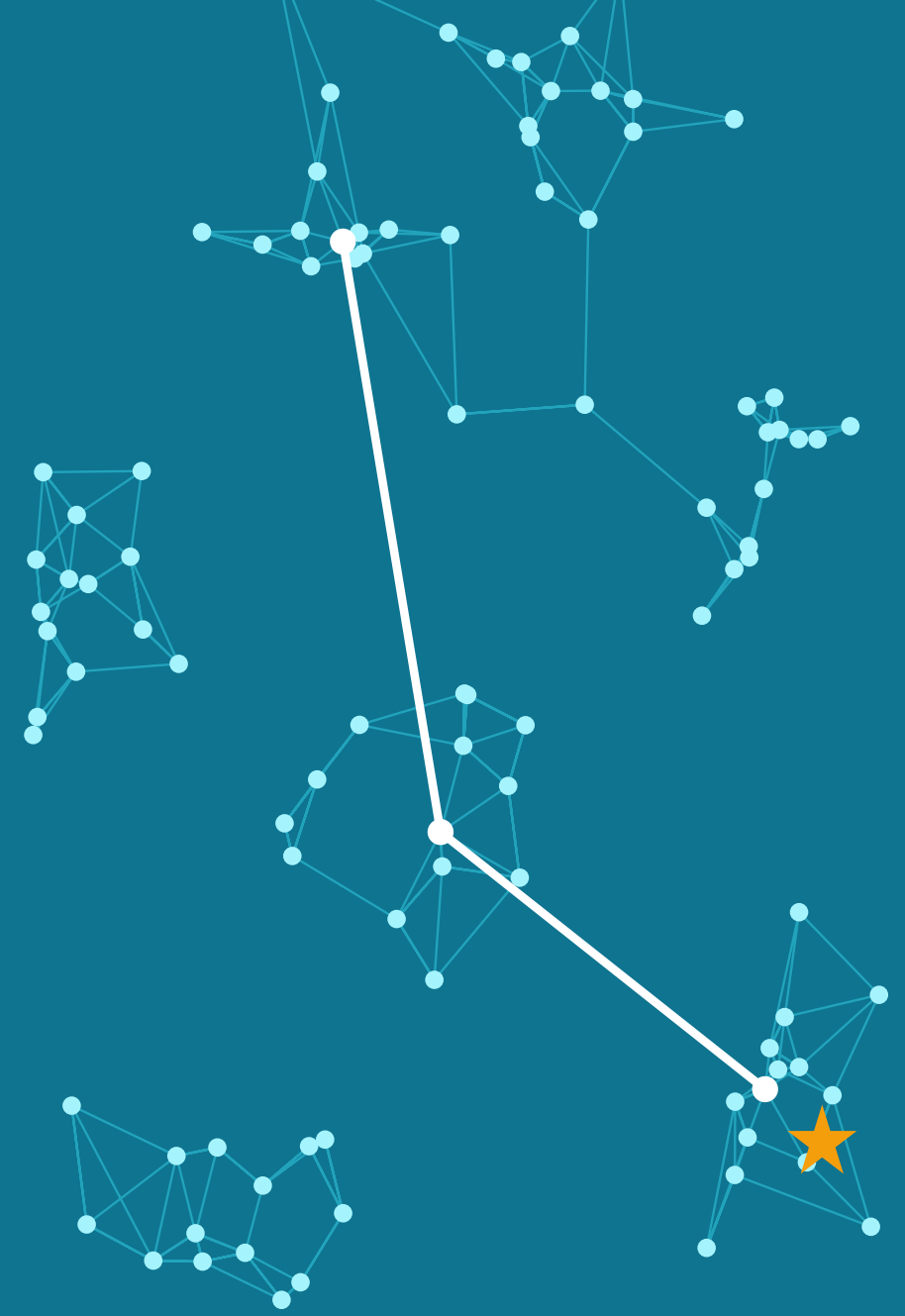
Small footprint

- Used ~26 MB on disk

LIVE DEMO

Demo 2: Performance at scale

Build and inspect the index · kNN vs ANN · 1,000,000 rows · read-only tables · the filter gotcha



Into production

Build a working application on all of it



Adding vectors grew the database 3.7×

StackOverflowScale: 1,000,000 posts, each with a 768-dimension float32 embedding

	Size	Note
Posts (the source text)	1,244 MB	1.57M rows
QuestionEmbeddings	4,110 MB	3,080-byte vector + 481-byte chunk per row, 2 rows per page
Vector index graph	261 MB	Small next to the vectors
All tables, with vectors	5,982 MB	1,611 MB without the vectors and graph
Compressed backup (ZSTD)	4,246 MB	1.41:1. Vectors are floats and barely compress

Read I/O

- Exact kNN: ~535,000 pages, 4.1 GB
- ANN: 5,564 pages

Write I/O

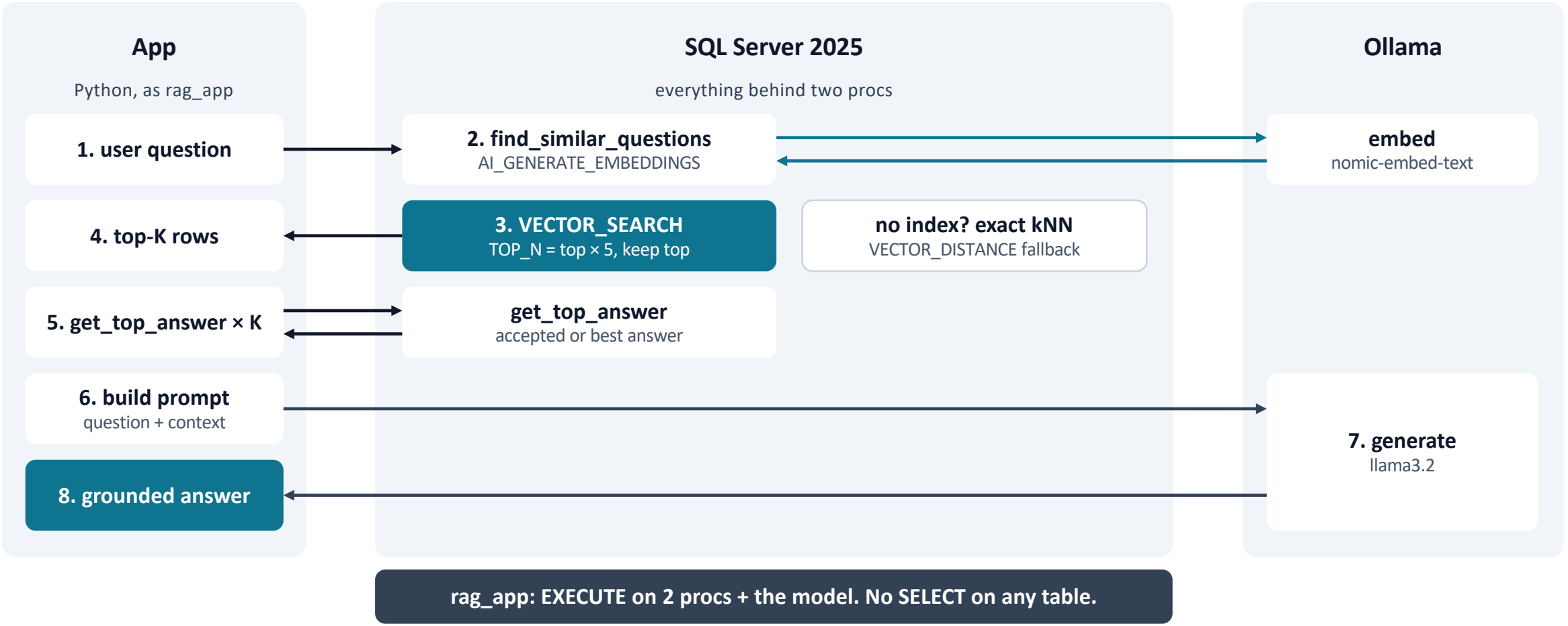
- ~3.6 KB of log per row
- 1M rows: ~3.6 GB of log per load

Plan for it

- Own filegroup, sized ahead
- Buffer pool, backups, restore time

RAG application architecture

The app sends text and gets rows back. It never sees a vector.



Patterns that make it production-ready

Satellite table

Vectors in their own table and filegroup.

- Single INT key (required on SQL Server 2025)
- Rebuild without touching Posts
- chunk = what was embedded

Procs as the contract

The app calls two procs.

- Exact fallback with no index
- Oversamples 5× for recall
- Swap models in one place

Least privilege

rag_app can EXECUTE, not SELECT.

- VIEW DEFINITION so the proc sees the index
- No access to the vectors
- Default database: StackOverflow

Operate it

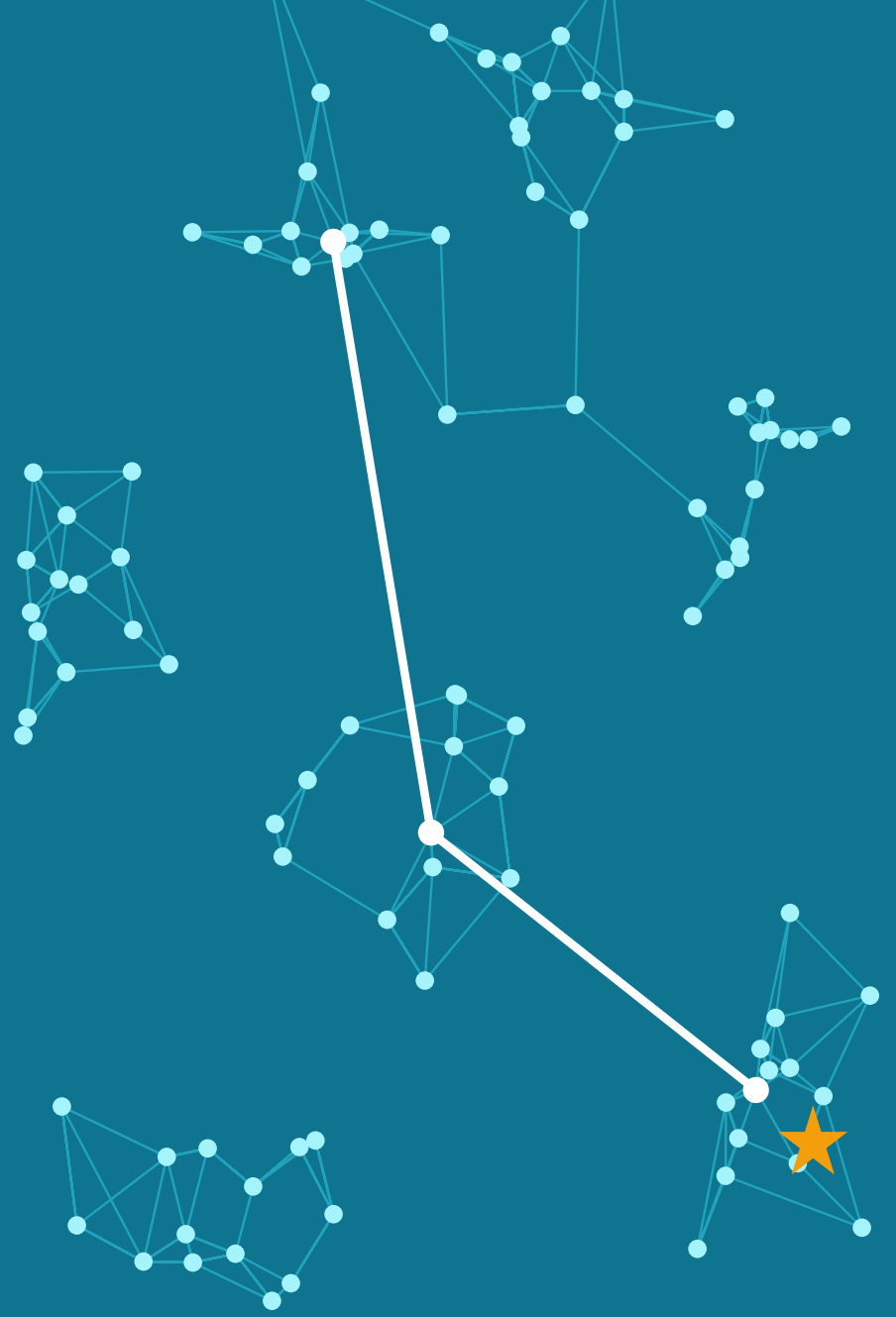
Drop, load, rebuild on a schedule.

- The fallback isn't free at 1M rows
- Change Tracking finds stale chunks
- Alert when the app reports kNN

LIVE DEMO

Demo 3: The RAG application

Schema · chunking · the search proc · drop, load, rebuild · a changed chunk · the app



Takeaways

Measure on your data, your model, your hardware

Storage

A VECTOR is a native float array in the row.

- float16 halves it for ~0.00001 of distance
- Embedding costs ~15 ms a row*

Indexes

Know where the line is.

- Exact search under ~50,000 vectors
- ANN at 1M rows: 5,564 reads, not 535,731
- Oversample to keep recall
- Filters run after the walk: oversample or get 0 rows

Production

Put it behind procs.

- Least privilege by default
- Plan for read-only tables on 2025
- Watch the fallback path

** Environment specific: nomic-embed-text on Ollama, Apple M2 GPU, through a local NGINX TLS proxy. Your model, hardware, and network will differ, so measure it.*

Questions?

github.com/nocentino/vector-under-the-hood

Every demo, the lab, and the app, running on your laptop

- Vector data type: learn.microsoft.com/sql/t-sql/data-types/vector-data-type
- CREATE VECTOR INDEX: learn.microsoft.com/sql/t-sql/statements/create-vector-index-transact-sql
- VECTOR_SEARCH: learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql
- DiskANN: Subramanya et al., NeurIPS 2019

Your feedback is important to us



Evaluate this session at:

passdatacommunitysummit.com/evaluations